CS 663 : Digital Image Processing : Assignment 2

Instructor : Suyash P. Awate

Note: The input data/images for a question are present in the corresponding “data/” subfolder.

Use of any generative-AI based tools (aiding plagiarism) is prohibited. If any such usage is found, it will be treated as a case of plagiarism. The difficulty level for all assignments in this course, including all underlying aspects of pre-processing, the main algorithm implementation, post-processing, and visualization is low enough that the use of sophisticated coding tools isn’t required. Needless to say, the goal of the assignment is to learn the image-processing algorithms, and NOT to learn to engineer appropriate prompts to get solutions from generative-AI tools.

5 points are reserved for submission in the described format.

When displaying grayscale images, use a colormap that uses at least 200 colors/intensities, and display the colormap/colorbar alongside that image.

Unless visualizing RGB images, don’t round the resulting images to integers; use a floating-point data type for the pixel value.

1. (20 points) Image Segmentation using Mean Shift.

   Input image 1: `data/segment/ghats.png`

   Input image 2: `data/segment/leopard.png`

   Input image 3: `data/segment/bison.png`

   (a) (5 points)

   Implement a function to perform mean-shift segmentation using pixel features that are a combination of pixel's RGB color and the pixel's XY coordinates. Tune all free parameters so that the method outputs a few segments (not more than 10 or so) per image.

   (b) (5 points)

   Implement a function that takes the output of the mean-shift method and creates a label image (segmentation map) where pixel values range from 1 to N, each indicating a segment out of a total of N segments. You may want to make your code computationally efficient by approximately modeling the feature-distribution using a Gaussian-kernel based nonparametric density using only a subset of pixels.

   Implement a function that takes two inputs, i.e., the RGB image and its label image (segmentation map), and creates an image that shows the segment boundaries (make them 2-pixel thick; black colored) drawn on the input image itself.

   (c) (5 points)

   Display the input image along with its segmentation (binary image) side by side.

   Also display another image where the segment boundaries are drawn on the input image itself.

2. (30 points) Anisotropic Diffusion, Bilteral Filtering.

   Input image 1: `data/diffusion/colorCheckerNoisy.png`

   Input image 2: `data/diffusion/rajaRaviVarma.png`

   Input image 3: `data/diffusion/sagarmatha.png`

   Input image 4: `data/diffusion/racing.png`

   (a) (12 points)

   Write a function to implement Perona-Malik anisotropic diffusion algorithm (exactly as discussed in class) for denoising the input images. Tune the parameters such that noise is visibly reduced and the major edges are preserved.

   (b) (12 points)

   Write a function to implement bilateral filtering algorithm (exactly as discussed in class) for denoising the input images. Tune the parameters such that noise is visibly reduced and the major edges are preserved.

   (c) (8 points)

   Display the original images, the noisy images, and the denoised images side-by-side.

   [Don't show small-sized images in the report. Make images as large as possible during visualization. Don't leave too much whitespace within the page. In addition, zoom into specific regions to clearly show the differences.]

3. (30 points) Texture Synthesis, Inpainting.

   Input image 1: `data/texture/textureSample81.png`

   Input image 2: `data/texture/textureSample110.png`

   Input image 3: `data/texture/textureSample207.png`

   Input image 4: `data/texture/lohagad.png`

   (a) (10 points)

   Write a function to implement the texture-synthesis algorithm (exactly as discussed in class) to synthesize a texture image of size 256×256 pixels. Place the provided seed image (textureSample*.png) in the center of the 256×256 pixels. Tune the parameters so that the synthesized texture appears as close as possible to the provided sample texture.

   [Make your algorithm efficient so that it runs within a minute or so.]

   (b) (4 points)

   Show the sample images (small; textureSample*.png) beside the synthesized images (256×256 pixels).

   (c) (10 points)

   Write a function to implement an inpainting algorithm to fill an arbitrary shaped region with missing data. Your algorithm should rely on the texture synthesis method (exactly as discussed in class).

   [Make your algorithm efficient so that it runs within a minute or so on the given image.]

   (d) (6 points)

   Apply your algorithm to remove the text in the image lohagad.png. Tune the parameters so that the synthesized texture appears as close as possible to the provided sample texture.

   When you remove the text, and create a region with missing data (“hole”), try to avoid removing too much of the non-text content present in the image. Design a suitable autmated/semi-automated algorithm for this purpose.

   Show the given image (with “hole”) beside the inpainted/hole-filled image.

4. (35 points) PCA for face recognition.

   Dataset: `data/ATT_Faces_Dataset.zip`

   (a) (5 points)

   Implement a function to compute the mean and the principal modes of variation for the set of images.

   For this purpose, use a training dataset comprising images 1-8 for subjects numbered 1-30 (total 240 images).

   • Show the mean image.

   • Show the first 10 principal modes of variation (using images) around the mean image. For each mode, show each eigenface image as well as the mean $\pm\alpha$ eigenface image.

   (b) (10 points)

   Implement a function to determine whether any given image lies close to the "face space" estimated from the training set or not. This determines whether a given image is a face or not.

   Design a score for this purpose (exactly as discussed in class lectures), which computes the distance of the given image from the "face space".

   Compute this score for all images, for each subject, in the dataset.

   Compute a matrix of such scores across all images in the dataset. This matrix M_1 will of size number-of-subjects $\times$ number-of-images-per-subject.

   (c) (10 points)

   Implement a function to recognize the person in a given image, when the image is assumed to be a face (i.e., image is assumed to lie sufficiently close to face space).

   Design a classifier for this purpose (exactly as discussed in class lectures), which computes the distance between (i) the face-space projection of the given face image and (ii) its nearest face-space projection across all face images in the training set.

   Apply this function to every image in the dataset (blindly/incorrectly assuming that every image passes the reconizablity test), and produce a matrix M_2 of size number-of-subjects $\times$ number-of-images-per-subject.

   (d) (10 points: 1 + 3 + 3 + 3)

   Use all remaining images, i.e., NOT in the training set, as the test set.

   Use the aforementioned functions to implement a method for face recognition. Apply it to the test set. For each image, the method should indicate one of the following three things: (i) the given image isn't a face image (output = -1); (ii) the given image is a face image but absent in the training

set (output = 0); (ii) the given image is a face image present in the training set, and the index of the specific person in the training set to which the given image belongs (output = index).

• Tune all free parameters to optimize the accuracy of face-recognition system on the test set. Report the accurracy on the test set, i.e., percent of correct decisions made for each image in the test set.

• For the tuned method, visualize/show matrix M_1 as an image, using the jet colormap.

• For the tuned method, visualize/show matrix M_2 as an image, using the jet colormap.

• Apply the tuned method to every image (training + test) in the dataset. Create a matrix M_3 of decision ouputs. Visualize/show matrix M_3 as an image, using the jet colormap.